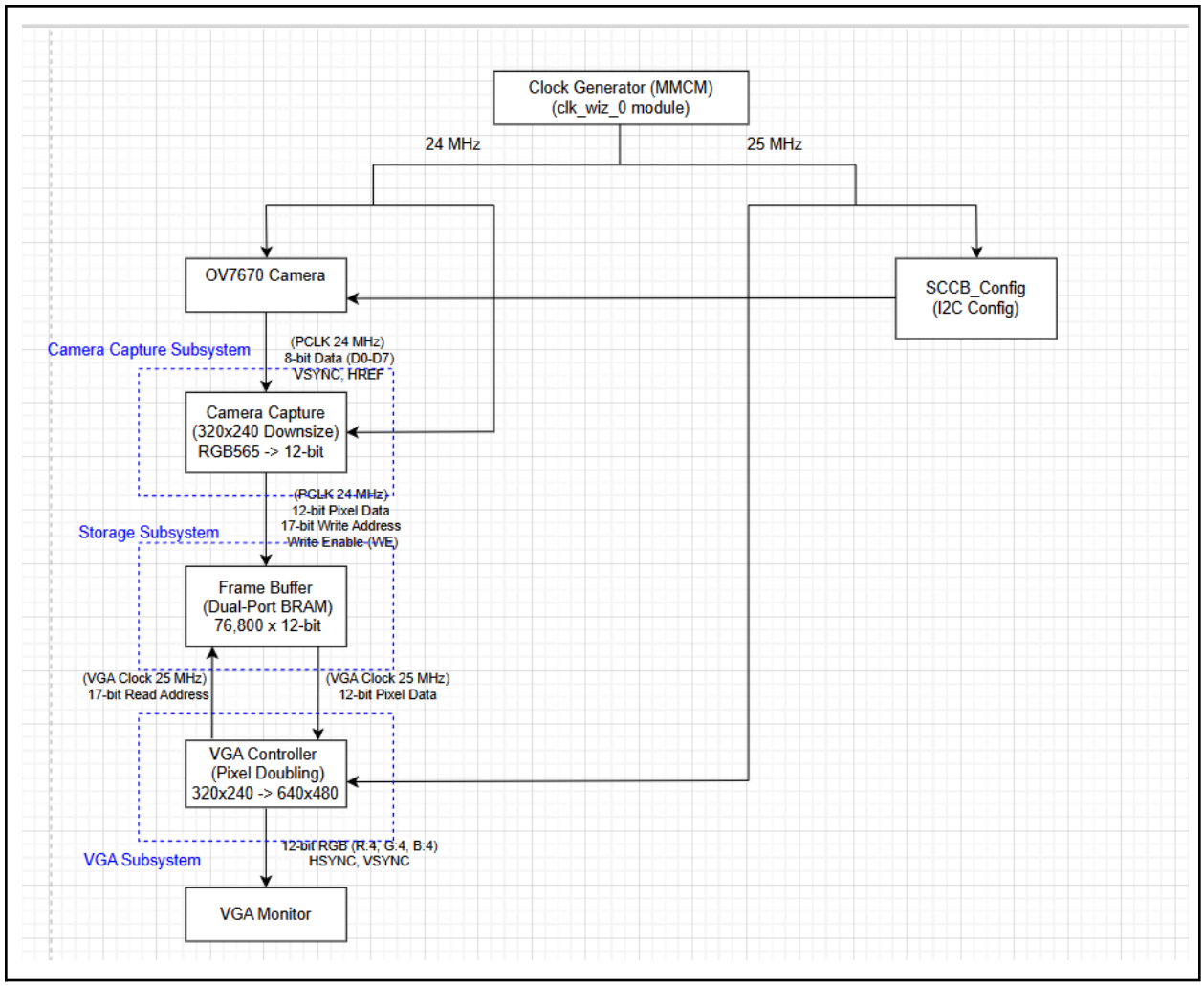
Group Member

Group name : ไม่ใช่ราชา"Hard"ware"

Name	Student Number
นายธนกฤต บรรลุประสงค์	6731321421
นายปาณัสม์ หล่อชัชวาลกุล	6731331721
นายวงศ์ฉัตร เกิดวิชัย	6731346121
นายศิวาพัชร์ ลิขิตธนพงศ์	6731354121

Overall Design Block Diagram

Block Diagram:



Original picture :
https://drive.google.com/file/d/1N1F1RvsEUcBtE0W0M58z3MniMEtaaghy/view?usp=sharing

Design Decision.

In this section, explain the reasoning behind your design choices. Consider the following aspects when justifying your implementation:

1. Module Architecture
    - Why did you structure your design this way?
        - This structure is based on the principle of modularity. Assigning a specific task to each module not only improves readability but also makes it much easier to isolate and debug individual components during testing.
    - What are the advantages of using this approach? (e.g., modularity, efficiency, ease of debugging)
        - It's easy to debug and implement because we divided the code into the sub-module.
2. Communication Protocols
    - Why did you choose this specific protocol (e.g., SPI, I2C, UART,AXI) for communication?
        - We use SCCB or I2C for controlling interfaces because it's suited for OV7670. We also use Parallel Video Bus for receiving the data from the camera to the FPGA.
    - How does this protocol suit your design requirements?
        - This protocol is suitable because it is the specific standard required to configure the OV7670 camera's settings, including resolution and color format."
3. Clock and Data Transfer Rate
    - Why did you select this specific clock frequency?
        - The **25 MHz** was selected because it is the standard pixel clock required for a 640x480 VGA display at 60Hz. The **24 MHz** was generated specifically to provide the required system clock (XCLK) for the OV7670 camera.
    - How does the data rate impact system performance?
        - Increasing the data rate enables higher resolutions, but it impacts system performance because a larger resolution demands more memory (BRAM) capacity.

- Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?
    - Yes, we considered these constraints. For **timing**, we managed the clock frequencies to match the specific requirements of the plugged-in devices (e.g., the OV7670 camera and VGA display). For **compatibility**, we selected the appropriate protocol (SCCB) for hardware configuration. Finally, regarding **noise margins**, operating at a relatively low clock speed reduces the chance of data reception errors, making the system less sensitive to noise.

4. Resource Utilization
    - How does your design balance resource usage (LUTs, FFs, BRAMs) and performance?
        - To maximize the limited BRAM, we use a single memory buffer where new data continuously overwrites the old addresses. This approach ensures the most efficient use of the available space possible.
    - Why did you optimize certain parts of the design?
        - We specifically optimized this part because unoptimized image data would easily exceed the board's memory limits, making it impossible to display the image properly.

Implementation Detail

Module : top_module.v

Module code :

```
`timescale 1ns / 1ps

module top_module(
    input clk_100mhz,
    input reset,
    input [1:0] sw,      // ใช้ Switch (00: Normal, 01: Filter)
    output ov7670_xclk,
    input ov7670_pclk,
    input ov7670_vsync,
    input ov7670_href,
    input [7:0] ov7670_data,
    output ov7670_sioc,
    inout ov7670_siod,
    output ov7670_pwdn,
    output ov7670_rst,
    // VGA Pins
    output [3:0] vga_r, vga_g, vga_b,
    output vga_hsync, vga_vsync
);
    // --- State Definition ---
    parameter S_NORMAL   = 2'b00;
    parameter S_GREY = 2'b01;
    parameter S_INVERSION = 2'b10;
    parameter S_CISOLATION = 2'b11;

    reg [1:0] current_state = S_NORMAL;

    // --- Signals ---
    wire clk_25m, clk_24m;
    wire [16:0] frame_addr;
    wire [11:0] pixel_12bit;
    wire [16:0] capture_addr; // [FIXED] ขนาดต้องเป็น 17-bit ให้ตรงกับ capture module
    wire [11:0] capture_data;
    wire capture_we;

    // สัญญาณสีที่จะส่งออก VGA จริงๆ
```

```
reg [3:0] r_out, g_out, b_out;

// --- Module Instantiations ---
clk_wiz_0 clock_gen (.clk_in1(clk_100mhz), .clk_out1(clk_25m), .clk_out2(clk_24m));
assign ov7670_xclk = clk_24m;
assign ov7670_pwdn = 1'b0;
assign ov7670_rst  = 1'b1;

sccb_config config_inst (.clk(clk_25m), .sioc(ov7670_sioc), .siod(ov7670_siod));

camera_capture_640x320 capture_inst (
    .pclk(ov7670_pclk), .vsync(ov7670_vsync), .href(ov7670_href),
    .d_in(ov7670_data), .addr_out(capture_addr), .data_out(capture_data), .write_en(capture_we)
);

frame_buffer_640x320 ram_inst (
    .clk_w(ov7670_pclk), .we(capture_we), .addr_w(capture_addr), .din(capture_data),
    .clk_r(clk_25m), .addr_r(frame_addr), .dout(pixel_12bit) // [FIXED] แก้จาก pixel_8bit เป็น pixel_12bit
);



wire [3:0] raw_r, raw_g, raw_b;
wire video_active;
vga_640x320_display vga_inst (
    .clk_25m(clk_25m), .pixel_in(pixel_12bit),
    .hsync(vga_hsync), .vsync(vga_vsync),
    .vga_r(raw_r), .vga_g(raw_g), .vga_b(raw_b),
    .frame_addr(frame_addr),
    .active(video_active)
);

// --- State Machine Logic ---
always @(posedge clk_25m) begin
    if (reset) begin
        current_state <= S_NORMAL;
    end else begin
        case (sw)
            2'b00: current_state <= S_NORMAL;
            2'b01: current_state <= S_GREY;
                    2'b10: current_state <= S_INVERSION;
```

```
                    2'b11: current_state <= S_CISOLATION; // [FIXED] แก้ typo จาก <+ เป็น <=
                default: current_state <= S_NORMAL;
            endcase
        end
    end

    // --- Grayscale Math (Shift-and-Add) ---
// Zero-extend to 8 bits to preserve precision during division/shifting
wire [7:0] R_ext = {raw_r, 4'b0000};
wire [7:0] G_ext = {raw_g, 4'b0000};
wire [7:0] B_ext = {raw_b, 4'b0000};

// Divide by powers of 2 using right shifts to approximate luminance
wire [7:0] Y_calc = (R_ext >> 2) + (R_ext >> 4) +
                    (G_ext >> 1) + (G_ext >> 4) +
                    (B_ext >> 3);

wire [3:0] gray_val = Y_calc[7:4];

always @(*) begin
    case (current_state)
        S_NORMAL: begin
            r_out = raw_r;
            g_out = raw_g;
            b_out = raw_b;
        end
        S_GREY: begin
            r_out = gray_val;
            g_out = gray_val;
            b_out = gray_val;
        end
        S_INVERSION: begin
            r_out = ~raw_r;
            g_out = ~raw_g;
            b_out = ~raw_b;
        end
        S_CISOLATION: begin
            // Isolate Red, force Green and Blue to 0
            r_out = raw_r;
            g_out = 4'h0;
```

```
            b_out = 4'h0;
        end
        default: begin
            r_out = raw_r;
            g_out = raw_g;
            b_out = raw_b;
        end
    endcase
end
    assign vga_r = video_active ? r_out : 4'h0;
    assign vga_g = video_active ? g_out : 4'h0;
    assign vga_b = video_active ? b_out : 4'h0;

endmodule
```

Module Description :

Topmodule initializes all sub modules and manages the display whether it is on or off and the three filters which are grey scale filter , inversion filter and red isolation channel.

AI disclosure:

We have used AI to implement the base module from the block diagram to make a finite state machine that manages the display. Later we add the filters by letting AI search how to make those filters.

Module : camera_capture_640x320.v

Module code :

```
module camera_capture_640x320(
    input pclk, vsync, href,
    input [7:0] d_in,

    // [แก้ไขตรงนี้] เติม = 0 ให้กับ output reg ทุกตัว
    output reg [16:0] addr_out = 0,
    output reg [11:0] data_out = 0,
    output reg write_en = 0
);

    reg [7:0] b1 = 0;             // เติม = 0
    reg byte_sel = 0;
```

```
reg [9:0] line_cnt = 0;
reg [9:0] pxl_cnt = 0;
reg old_href = 0;

always @(posedge pclk) begin
    old_href <= href;

    if (vsync) begin
        addr_out <= 0;
        line_cnt <= 0;
        pxl_cnt <= 0;
        byte_sel <= 0;
        write_en <= 0;
    end else if (href) begin
        if (byte_sel == 0) begin
            b1 <= d_in;
            byte_sel <= 1;
            write_en <= 0;
        end else begin
            // Downsample: เก็บเฉพาะบรรทัดคู่และพิกเซลคู่ (ย่อ 640x480 -> 320x240)
            if (line_cnt[0] == 0 && pxl_cnt[0] == 0 && addr_out < 76800) begin
                data_out <= {b1[7:4], b1[2:0], d_in[7], d_in[4:1]};
                write_en <= 1;
                addr_out <= addr_out + 1;
            end else begin
                write_en <= 0;
            end
            pxl_cnt <= pxl_cnt + 1;
            byte_sel <= 0;
        end
    end else begin
        write_en <= 0;
        pxl_cnt <= 0;
        byte_sel <= 0;

        if (old_href == 1 && href == 0) begin
            line_cnt <= line_cnt + 1;
        end
    end
end
```

```
endmodule
```

Module Description : Manage the data that input from the OV7670 by receiving 1 byte per clock it will wait for 2 bytes before downsample from the b1 and d_in in RGB 565 change it to RGB 444 and send the address in the pixel and ensure it doesn't exceed the 320*240 pixels
AI disclosure: USe to implement the detail to receive the 8 bytes per second input and transform into RGB444 with constraints

Module : frame_buffer_640x320.v
Module code :

```
`timescale 1ns / 1ps

module frame_buffer_640x320 (
    input clk_w,
    input we,
    input [16:0] addr_w,
    input [11:0] din,
    input clk_r,
    input [16:0] addr_r,
    output reg [11:0] dout = 12'h000 // [แก้ตรงนี้] เติม = 12'h000 เพื่อตั้งค่าเริ่มต้นให้ register
);

    // 1. ประกาศหน่วยความจำ (204,800 ช่อง สำหรับ 640x320)
    reg [11:0] mem [0:204799];

    // ----------------------------------------------------------
    // 2. ล้างค่า X ให้เป็น 0 ในหน่วยความจำ
    // ----------------------------------------------------------
    integer i;
    initial begin
        for (i = 0; i < 204800; i = i + 1) begin
            mem[i] = 12'h000;
        end
    end
    // ----------------------------------------------------------

    // 3. ส่วนการเขียน (Write Port)
    always @(posedge clk_w) begin
```

```
    if (we) begin
        mem[addr_w] <= din;
    end
  end

  // 4. ส่วนการอ่าน (Read Port)
  always @(posedge clk_r) begin
    dout <= mem[addr_r];
  end

endmodule
```

Module Description :This module acts as a video frame buffer to store image pixel data. It is implemented as a Simple Dual-Port Block RAM (BRAM) with a capacity of 76,800 words, corresponding to a 320x240 image resolution with a 12-bit color depth per pixel.

AI disclosure:Use to config the BRAM

Module : sccb_config.v

Module code :

```
module sccb_config(
  input clk,          // Master Clock (เช่น 25MHz)
  output sioc,        // SIO_C (Clock)
  inout siod          // SIO_D (Data)
);
  // ------------------------------------------------------
  // 1. Clock Divider: สร้าง Base Clock (~400 kHz)
  // สำหรับ 4-phase state machine เพื่อสร้าง SCCB Clock ที่ ~100 kHz
  // ------------------------------------------------------
  reg [7:0] clk_div = 0;
  reg i2c_clk = 0;
  always @(posedge clk) begin
    if (clk_div == 31) begin // 25MHz / (32 * 2) = ~390 kHz
      clk_div <= 0;
      i2c_clk <= ~i2c_clk;
    end else begin
      clk_div <= clk_div + 1;
    end
  end
```

```
// ----------------------------------------------------------
// 2. Register ROM: ตารางเก็บค่าตั้งค่ากล้อง
// ----------------------------------------------------------
reg [7:0] reg_idx = 0;
reg [15:0] reg_data;        // [15:8] = Sub-address, [7:0] = Write Data
wire [7:0] TOTAL_REGS = 8; // [FIXED] กลับไปใช้ 8 คำสั่งพื้นฐานที่เสถียรที่สุด

always @(*) begin
    case(reg_idx)
        0: reg_data = 16'h1280; // COM7: RESET
        1: reg_data = 16'h1101; // CLKRC: Enable clock prescaler
        2: reg_data = 16'h6B4A; // DBLV: Stabilize PLL clock
        3: reg_data = 16'h1204; // COM7: RGB Mode
        4: reg_data = 16'h8C00; // RGB444: Disable
        5: reg_data = 16'h40D0; // COM15: RGB565 Full range
        6: reg_data = 16'h3A04; // TSLB
        7: reg_data = 16'hB084; // Magic Register
        default: reg_data = 16'hFFFF;
    endcase
end

// ----------------------------------------------------------
// 3. SCCB State Machine (4-Phase ป้องกัน Race Condition)
// ----------------------------------------------------------
reg [3:0] state = 0;
reg [5:0] bit_cnt = 0;
reg [23:0] shift_reg = 0;   // เก็บ {ID Address, Sub-address, Write Data}
reg sioc_reg = 1;
reg siod_reg = 1;
reg siod_out_en = 1;        // 1 = FPGA ขับสัญญาณ, 0 = High-Z (ช่วง Don't-Care bit)
reg [9:0] delay_cnt = 0;    // 10-bit สำหรับนับ 400

wire [7:0] ID_ADDR = 8'h42; // ID Address ของกล้อง (บิตที่ 0 เป็น 0 = Write)

assign sioc = sioc_reg;
// ป้องกันการชนกันของสัญญาณ (Bus Contention) ด้วย Open-Drain
assign siod = (siod_out_en && siod_reg == 1'b0) ? 1'b0 : 1'bz;

always @(posedge i2c_clk) begin
    case (state)
```

```
0: begin // IDLE: เตรียมข้อมูล
   if (reg_idx < TOTAL_REGS) begin
      // รอ 1ms หลังจากสั่ง Reset กล้อง (คำสั่งที่ 0)
      // i2c_clk ~400kHz (2.5us), ดังนั้น 1ms = 400 cycles
      if (reg_idx == 1 && delay_cnt < 400) begin
         delay_cnt <= delay_cnt + 1;
      end else begin
         shift_reg <= {ID_ADDR, reg_data[15:8], reg_data[7:0]};
         siod_out_en <= 1;
         sioc_reg <= 1;
         siod_reg <= 1;
         state <= 1;
      end
   end
end

1: begin // START Condition (SIO_D ลง 0 ขณะที่ SIO_C เป็น 1)
   siod_reg <= 0;
   bit_cnt <= 23;
   state <= 2;
end

// --- 4-Phase Send Bit ---
2: begin // Phase 0: ดึง Clock ลง LOW
   sioc_reg <= 0;
   state <= 3;
end

3: begin // Phase 1: เปลี่ยนข้อมูล (ขณะ Clock เป็น LOW ปลอดภัย)
   siod_reg <= shift_reg[bit_cnt];
   state <= 4;
end

4: begin // Phase 2: ดึง Clock ขึ้น HIGH
   sioc_reg <= 1;
   state <= 5;
end

5: begin // Phase 3: ค้าง Clock เป็น HIGH ให้ Slave อ่าน
   if (bit_cnt == 16 || bit_cnt == 8 || bit_cnt == 0) begin
```

```
            state <= 6; // ส่งครบ 8 บิต ไป Don't Care
        end else begin
            bit_cnt <= bit_cnt - 1;
            state <= 2; // ส่งบิตต่อไป
        end
    end

    // --- 4-Phase Don't Care Bit ---
    6: begin // Phase 0: ดึง Clock ลง LOW
        sioc_reg <= 0;
        state <= 7;
    end

    7: begin // Phase 1: ปล่อยสายเป็น High-Z ให้ Slave ตอบ ACK
        siod_out_en <= 0;
        state <= 8;
    end

    8: begin // Phase 2: ดึง Clock ขึ้น HIGH
        sioc_reg <= 1;
        state <= 9;
    end

    9: begin // Phase 3: ค้าง Clock เป็น HIGH
        if (bit_cnt == 0) begin
            state <= 10; // จบ Phase 3 แล้ว เตรียม STOP
        end else begin
            bit_cnt <= bit_cnt - 1;
            state <= 13; // กลับไปส่งข้อมูลต่อ
        end
    end

    13: begin // ดึง Clock ลง LOW ก่อน
        sioc_reg <= 0;
        state <= 14;
    end

    14: begin // Master กลับมาขับสาย SIO_D
        siod_out_en <= 1;
        state <= 3; // ไปเปลี่ยนข้อมูลเป็นบิตถัดไป (ข้าม state 2)
```

```
        end

        // --- STOP Condition ---
        10: begin // เตรียม STOP: ดึง Clock ลง LOW
            sioc_reg <= 0;
            state <= 11;
        end

        11: begin // Master ขับ Data ให้เป็น 0 ขณะ Clock เป็น LOW
            siod_out_en <= 1;
            siod_reg <= 0;
            state <= 12;
        end

        12: begin // ดึง Clock ขึ้น HIGH
            sioc_reg <= 1;
            state <= 15;
        end

        15: begin // ดึง Data ขึ้น HIGH ขณะ Clock เป็น HIGH -> STOP
            siod_reg <= 1;
            reg_idx <= reg_idx + 1; // เลื่อนไปรีจิสเตอร์ถัดไป
            state <= 0;
        end

        default: state <= 0;
      endcase
    end
endmodule
```

Module Description :This module initializes the OV7670 camera by writing predefined configuration values to its internal registers. It implements an SCCB (I2C-like) master controller using a 4-phase state machine to ensure stable data transmission without bus contention.
AI disclosure:Use AI to find the OV7670 register to configure and use it to write all of the options so that the camera could work and display correctly.

Module : vga_640x320_display.v
Module code :

```
module vga_640x320_display(
    input clk_25m,
    input [11:0] pixel_in,
    output hsync, vsync,
    output [3:0] vga_r, vga_g, vga_b,
    output [16:0] frame_addr,
    output active
);
    reg [9:0] h_cnt = 0, v_cnt = 0;

    always @(posedge clk_25m) begin
        if (h_cnt == 799) begin
            h_cnt <= 0;
            v_cnt <= (v_cnt == 524) ? 0 : v_cnt + 1;
        end else h_cnt <= h_cnt + 1;
    end
    assign hsync = (h_cnt >= 656 && h_cnt < 752) ? 0 : 1;
    assign vsync = (v_cnt >= 490 && v_cnt < 492) ? 0 : 1;

    wire display_area = (h_cnt < 640 && v_cnt < 480);

    // [FIXED] เพิ่ม Delay 1 Clock ให้สัญญาณ active เพื่อให้ตรงกับความหน่วงของ RAM
    reg active_d = 0;
    always @(posedge clk_25m) active_d <= display_area;
    assign active = active_d;

    // [FIXED] ใช้เทคนิคการยืดภาพ (Scaling) แบบเนียนตา
    // เรามีภาพจริง 312 พิกเซลใน RAM (ข้าม 8 พิกเซลแรกที่เป็นแถบดำ)
    // เราจะยืด 312 พิกเซลนี้ให้เต็ม 640 พิกเซลบนจอ ด้วยสมการ: h_cnt * (312/640)
    // ประมาณค่าคณิตศาสตร์แบบไม่ใช้การหาร: (h_cnt * 499) / 1024
    wire [19:0] scaled_x = (h_cnt * 499) >> 10;
    wire [9:0] img_x = scaled_x + 8; // เริ่มอ่านที่ Index 8 เพื่อข้ามแถบดำ
    wire [9:0] img_y = v_cnt >> 1;

    // คำนวณ Address จากภาพขนาด 320x240
    assign frame_addr = display_area ? (img_y * 320 + img_x) : 0;

    assign vga_r = active_d ? pixel_in[11:8] : 4'h0;
    assign vga_g = active_d ? pixel_in[7:4] : 4'h0;
    assign vga_b = active_d ? pixel_in[3:0] : 4'h0;
```

```
endmodule
```

Module Description :The `vga_640x320_display` module acts as a VGA Display Controller. It generates standard 640x480 @ 60Hz VGA timing signals (HSYNC, VSYNC). Furthermore, it handles image scaling by mapping a 320x240 image stored in the Frame Buffer to fit the 640x480 display area. It calculates the correct memory address for each pixel, synchronizes the memory read latency, and outputs the 12-bit RGB video signals.

AI disclosure:Use this to generate this code base on the block diagram to make it can display on 640 * 320 with the correct data later fixed so that it can be upscaled.

Challenge Faced

1. There is a lot of documentation to read and the documentation is detailed.
2. Don’t know where to start.
3. Can not vibe code in one attempt.
4. Take too much time in each iteration (Generate bitstream too long)
5. Sometimes it is hard to debug.
6. Hard to test.